Lesson 3 Worksheet: Lists — ANSWER KEY

Part 1: Creating Lists

Description	Code
An empty list	`path = []`
A list with three numbers: 1, 2, 3	`numbers = [1, 2, 3]`
A list with two strings: "hello", "world"	`words = ["hello", "world"]`
A list with one tuple: (0, 0)	`path = [(0, 0)]`

What characters surround a list's values? Square brackets `[ ]`

Tuple uses ( ) and list uses [ ]. Why does the difference matter?

Tuples are immutable (cannot be changed after creation), while lists are mutable (you can add, remove, or change items). We use tuples for fixed data like a single coordinate and lists when we need to build up or modify a collection, like a path of coordinates.

Part 2: List vs. Tuple

Feature	Tuple	List
Bracket type	( )	[ ]
Can you change it after creation?	No (immutable)	Yes (mutable)
Best used for	Fixed data like a coordinate	A collection that grows or changes, like a path
Example	`(2, 3)`	`[1, 2, 3]`

Part 3: Append Exercises

After line	Contents of `path`
`path = []`	`[]`
`path.append((0, 0))`	`[(0, 0)]`
`path.append((1, 0))`	`[(0, 0), (1, 0)]`
`path.append((2, 0))`	`[(0, 0), (1, 0), (2, 0)]`

How many items are in the list at the end? 3

What does `path[0]` return? (0, 0)

What does `path[2]` return? (2, 0)

What does `len(path)` return? 3

Part 4: Iteration Tracing

```
colors = ["red", "green", "blue"]

for color in colors:
    print(color)
```

Output:

1. red
2. green
3. blue

```
path = [(0, 0), (1, 0), (2, 0)]

for step in path:
    print("Visit:", step)
```

Output:

1. Visit: (0, 0)
2. Visit: (1, 0)
3. Visit: (2, 0)

Part 5: Accessing Tuples Inside a List

```
path = [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2)]
```

Expression	Value
`path[0]`	(0, 0)
`path[3]`	(2, 1)
`path[0][0]`	0
`path[0][1]`	0
`path[3][0]`	2
`path[3][1]`	1
`len(path)`	5
`path[-1]`	(2, 2)

Explain what `path[3][1]` means in words:

Get the 4th item in the path list (index 3), which is the tuple (2, 1), then get the second value in that tuple (index 1), which is 1. This gives the column value of the 4th position in the path.

Part 6: Building a Path Step by Step

```
path = []
path.append((0, 0))    # Starting position
path.append((1, 0))    # One row down
path.append((2, 0))    # Two rows down
path.append((2, 1))    # One column right
path.append((2, 2))    # Two columns right
path.append((2, 3))    # Three columns right (destination)
```

Write out the final path list:

`path = [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]`

Part 7: Loop-Based Path Building

```
path = []
for row in range(4):
    path.append((row, 0))
print(path)
```

Iteration	row	Appended	List after append
1	0	(0, 0)	[(0, 0)]
2	1	(1, 0)	[(0, 0), (1, 0)]
3	2	(2, 0)	[(0, 0), (1, 0), (2, 0)]
4	3	(3, 0)	[(0, 0), (1, 0), (2, 0), (3, 0)]

Final output: [(0, 0), (1, 0), (2, 0), (3, 0)]

```
path = []
for col in range(3):
    path.append((2, col))
print(path)
```

Iteration	col	Appended	List after append
1	0	(2, 0)	[(2, 0)]
2	1	(2, 1)	[(2, 0), (2, 1)]
3	2	(2, 2)	[(2, 0), (2, 1), (2, 2)]

Final output: [(2, 0), (2, 1), (2, 2)]

Part 8: Iterating Through a Path

```
path = [(0, 0), (1, 0), (1, 1), (1, 2)]

print("Path has", len(path), "positions")

for step in path:
    row = step[0]
```

```
    col = step[1]
    print("Row:", row, "Col:", col)
```

Output:

1. Path has 4 positions
2. Row: 0 Col: 0
3. Row: 1 Col: 0
4. Row: 1 Col: 1
5. Row: 1 Col: 2

Part 9: Spot the Error

Error 1:

```
path = []
for i in range(3):
    position = (i, 0)
path.append(position)    # Something is wrong with the indentation
```

Problem: The `path.append(position)` line is not indented inside the for loop. Because it is outside the loop, it only runs once after the loop finishes, so only the last position (2, 0) gets appended to the list.

Fix: Indent `path.append(position)` so it is inside the for loop:

```
path = []
for i in range(3):
    position = (i, 0)
    path.append(position)
```

Error 2:

```
path = []
print(path[0])
```

Problem: IndexError – the list is empty (`[]`), so there is no item at index 0. You cannot access an element in an empty list.

Fix: Add an item to the list before trying to access it, or check that the list is not empty first:

```
path = []
path.append((0, 0))
print(path[0])
```

Error 3:

```
path = ((0, 0), (1, 0), (2, 0))   # Storing a path
path.append((3, 0))
```

Problem: `path` is a tuple of tuples (uses parentheses), not a list. Tuples are immutable and do not have an `append` method. This will cause an AttributeError.

Fix: Use square brackets to make `path` a list instead of a tuple:

```
path = [(0, 0), (1, 0), (2, 0)]
path.append((3, 0))
```

Part 10: Code Writing

```
path = []

for row in range(5):
    path.append((row, 0))

for step in path:
    print(step)
```

Reflection

A tuple is ONE position. A list of tuples is a PATH. Explain in your own words why we need both data structures.

Sample answer: A tuple holds a single coordinate like (2, 3) that should not change – row 2, column 3 is a fixed location on the grid. A list holds a sequence of those coordinates that represents the robot's path from start to destination. We need the list to be mutable so we can build the path one step at a time using `append()`. We need tuples to be immutable so each position stays fixed and reliable. Together, a list of

tuples lets us represent an ordered path of unchangeable positions – exactly what the robot needs to navigate the grid.